SMART HOME / .NET 8

Лабораторна робота №4

Розробка вебсерверів засобами ASP.NET Core

Виконав Фесенко Євген
Група 404-ТН

Мета роботи

Розробити REST API для кімнат і пристроїв SmartHome. Повторно використати асинхронні сервіси, репозиторії та SQLite-моделі Lab3. Перевірити CRUD через HTTP і створити вебінтерфейс для роботи з даними.

Архітектура

Рівень	Роль у SmartHome
HTTP / представлення	SmartHome.REST приймає JSON, перевіряє моделі запитів і формує HTTP-відповіді.
Прикладний сервіс	ICrudServiceAsync<T> і EfCrudService<T> реалізують асинхронний CRUD та пагінацію.
Доступ до даних	IRepository<T>, Repository<T> та SmartHomeContext працюють із SQLite через EF Core і міграцію Lab3.

Запуск із кореня репозиторію

```
dotnet build SmartHome.sln
dotnet test SmartHome.sln
dotnet run --project Lab4/SmartHome.REST
dotnet run --project Lab4/SmartHome.MVC
```

Запускайте REST і MVC в окремих терміналах. У режимі Development документація API доступна за адресою **http://localhost:5083/swagger**, а MVC за адресою **http://localhost:5084**. Обидва застосунки типово використовують **Lab4/SmartHome.Lab4.db**. Для окремої бази задайте **ConnectionStrings__SmartHome** для відповідного процесу.

Окрема SQLite-база створилася після застосування міграції **20261014103419_InitialCreate**. Swagger UI та документ OpenAPI 3.0.1 повернули HTTP 200. Документ описує п’ять операцій для кімнат і п’ять для пристроїв.

Після відновлення пакетів рішення зібралося без помилок і попереджень. Усі **7 із 7** наявних тестів xUnit пройшли.

Через HTML-форми MVC перевірено створення, перегляд списку й деталей, редагування та видалення кімнат і пристроїв на окремій SQLite-базі. Форма відхилила неіснуючу кімнату, а запит без токена захисту від CSRF отримав HTTP 400.

ПЕРЕВІРКА / HTTP

Результати запитів

31 із 31 HTTP-перевірок пройдено. Запити виконано скриптом PowerShell до локального сервера Kestrel на окремій SQLite-базі. Перші два запити підтвердили, що таблиці кімнат і пристроїв порожні.

У таблиці {id} позначає ідентифікатор із відповіді API. Фактичні ідентифікатори наведено в прикладах нижче.

№	HTTP	Маршрут	Сценарій	Код
1	GET	/api/rooms	Порожній список кімнат	200
2	GET	/api/devices	Порожній список пристроїв	200
3	POST	/api/rooms	Створення кімнати	201
4	GET	/api/rooms/{id}	Читання кімнати за Location	200
5	POST	/api/rooms	Створення другої кімнати	201
6	GET	/api/rooms?page=1&amount=1	Кімнати, сторінка 1	200
7	GET	/api/rooms?page=2&amount=1	Кімнати, сторінка 2	200
8	GET	/api/rooms?page=0&amount=1	Нульовий номер сторінки	400
9	GET	/api/rooms?page=2147483647&amount=100	Надто великий зсув сторінки	400
10	POST	/api/rooms	Порожня назва кімнати	400
11	PUT	/api/rooms/{id}	Оновлення кімнати	204
12	GET	/api/rooms/{id}	Перевірка збереження змін	200
13	POST	/api/devices	Відсутнє поле isOnline	400
14	POST	/api/devices	Невідома кімната пристрою	400
15	POST	/api/devices	Створення пристрою	201
16	GET	/api/devices/{id}	Читання пристрою за Location	200
17	GET	/api/devices?page=1&amount=1	Пристрої, сторінка 1	200
18	GET	/api/devices?page=2&amount=1	Порожня сторінка пристроїв	200
19	GET	/api/devices?page=1	Відсутній параметр amount	400
20	PUT	/api/devices/{id}	Оновлення з невідомою кімнатою	400
21	PUT	/api/devices/{id}	Оновлення пристрою	204
22	GET	/api/devices/{id}	Перевірка збереження змін	200
23	DELETE	/api/devices/{id}	Видалення пристрою	204
24	GET	/api/devices/{id}	Читання видаленого пристрою	404
25	DELETE	/api/devices/{id}	Повторне видалення пристрою	404
26	DELETE	/api/rooms/{id}	Видалення кімнати	204
27	GET	/api/rooms/{id}	Читання видаленої кімнати	404
28	PUT	/api/rooms/{id}	Оновлення видаленої кімнати	404
29	POST	/api/devices	Створення пристрою для перевірки каскаду	201
30	DELETE	/api/rooms/{id}	Видалення другої кімнати	204
31	GET	/api/devices/{id}	Перевірка каскадного видалення	404

Перевірено також заголовок Location після створення, фактичні дані після оновлення та відсутність ресурсу після видалення.

ПЕРЕВІРКА / ПРИКЛАДИ

Запити та відповіді

Витяги з того самого HTTP-запуску. JSON відформатовано для читання, а у відповіді про помилку наведено основні поля.

Створення кімнати

Запит

```
POST /api/rooms
{
  "name": "Living Room",
  "floor": 1
}
```

Відповідь

```
HTTP 201 Created
{
  "id": "1b345bc6-5b7f-4f8b-9f5a-5b5ec08be3de",
  "name": "Living Room",
  "floor": 1
}

Location: http://127.0.0.1:54792/api/rooms/1b345bc6-5b7f-4f8b-9f5a-5b5ec08be3de
```

Створення пристрою

Запит

```
POST /api/devices
{
  "isOnline": true,
  "name": "Lamp",
  "roomId": "1b345bc6-5b7f-4f8b-9f5a-5b5ec08be3de"
}
```

Відповідь

```
HTTP 201 Created
{
  "id": "d56d06b8-4200-4ee6-a80b-db2702c1974c",
  "name": "Lamp",
  "isOnline": true,
  "roomId": "1b345bc6-5b7f-4f8b-9f5a-5b5ec08be3de"
}

Location: http://127.0.0.1:54792/api/devices/d56d06b8-4200-4ee6-a80b-db2702c1974c
```

Некоректний номер сторінки

Запит

```
GET /api/rooms?page=0&amount=1
```

Відповідь

```
HTTP 400 Bad Request
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Page": [
      "The field Page must be between 1 and 2147483647."
    ]
  }
}
```

Видалення пристрою й повторне читання

```
DELETE /api/devices/d56d06b8-4200-4ee6-a80b-db2702c1974c
HTTP 204 No Content

GET /api/devices/d56d06b8-4200-4ee6-a80b-db2702c1974c
HTTP 404 Not Found
```

Контрольні запитання

1. Чим Web API відрізняється від MVC і Blazor?

Шаблон ASP.NET Core Web API налаштовує HTTP-маршрути й серіалізацію, тому контролери SmartHome.REST повертають JSON і коди HTTP. У MVC контролери можуть повертати Razor-представлення, що рендеряться в HTML. Blazor будує інтерфейс із Razor-компонентів і підтримує серверні та клієнтські режими інтерактивності.

2. Як реалізовано три рівні?

Контролери й DTO відповідають за HTTP. EfCrudService<T> реалізує операції над сутностями через IRepository<T>. Repository<T> та SmartHomeContext звертаються до SQLite. Поділ дозволяє змінювати HTTP-контракт і перевіряти окремі рівні незалежно.

3. Для чого окремі моделі в папці Models?

Моделі запитів визначають поля, які клієнт може передати. Моделі відповідей визначають поля, які клієнт отримає. Наприклад, ідентифікатор кімнати повертається у відповіді, а під час створення сервер генерує його сам. EF-сутності окремо описують схему, навігаційні властивості та зв’язки.

4. Що таке REST і як API дотримується однорідного інтерфейсу?

REST описує взаємодію навколо ресурсів та їхніх представлень. У SmartHome колекції мають сталі URI /api/rooms та /api/devices. Ідентифікатор у шляху адресує один ресурс. HTTP-метод виражає дію, JSON є представленням ресурсу, а статус і заголовки передають результат.

5. Які методи використовуються для CRUD?

POST створює ресурс у колекції. GET читає список або елемент. PUT оновлює елемент за ідентифікатором. DELETE видаляє елемент. Той самий набір методів реалізовано для кімнат і пристроїв.

6. Які HTTP-коди повертає API?

201 Created із Location після створення, 200 OK під час успішного читання, 204 No Content після оновлення або видалення. 400 Bad Request позначає невалідні дані або параметри, а 404 Not Found означає, що ресурс за URI відсутній. 409 Conflict передбачено на випадок конфлікту під час створення.

Контрольні запитання

7. Для чого ICrudServiceAsync<T>?

Інтерфейс задає спільний контракт для створення, читання одного або багатьох елементів, читання сторінки, оновлення, видалення та збереження. Під час асинхронного очікування операції введення-виведення потік обробки запитів може бути доступний для іншої роботи. Провайдер Microsoft.Data.Sqlite виконує введення-виведення синхронно. Запис змін виконує репозиторій. Microsoft Learn.

8. Як працює впровадження залежностей?

Контролер оголошує залежності в конструкторі. Контейнер ASP.NET Core підставляє EfCrudService<T> для ICrudServiceAsync<T> і Repository<T> для IRepository<T>. Реєстрація scoped разом з AddDbContext забезпечує один контекст у межах HTTP-запиту.

9. Як реалізовано пагінацію?

Запит із page та amount передає номер сторінки і її розмір до ReadAllAsync(page, amount). Номер починається з 1, а розмір становить від 1 до 100. Сервіс завантажує записи, сортує їх за Id і бере потрібне вікно в пам’яті через Skip і Take. Пагінація обмежує розмір HTTP-відповіді та дає змогу клієнту отримувати результати порціями.

10. Як перевірено API і що внесено у звіт?

HTTP-скрипт перевірив створення, читання, оновлення й видалення обох ресурсів, заголовок Location, пагінацію, валідацію та запити до видалених записів. У звіті наведено фактичні HTTP-коди й приклади тіл запитів і відповідей.

11. Як створено SmartHome.MVC?

Застосунок створено за шаблоном ASP.NET Core MVC. Інструмент dotnet-aspnet-codegenerator 8.0.22 з провайдером SQLite згенерував RoomsController, DevicesController і по п’ять Razor-представлень для кожної сутності. Представлення з папки Views формують HTML, а форми надсилають дані в дії контролерів.

12. Як MVC використовує моделі та контекст Lab3?

Проєкт SmartHome.MVC посилається на SmartHome.Infrastructure. Дії контролерів працюють із сутностями Lab3 через впроваджений SmartHomeContext. Застосунок використовує SQLite і застосовує наявну міграцію під час запуску.

Висновок

SmartHome має REST API для кімнат і пристроїв та MVC-інтерфейс. Обидва застосунки повторно використовують моделі й міграції Lab3. HTTP-перевірки підтвердили CRUD і обробку помилкових запитів.